

Instructions for the Teaching Assistant

This document provides instructions for testing the system, outlines optional features implemented, and describes the development and deployment process.

Implemented Optional Features

The following optional features have been implemented:

- Ability to change system state through HTTP requests (INIT, RUNNING, PAUSED).
- Service for retrieving requests (/request).
- Service for retrieving run logs (/run-log).
- Static analysis step included in the CI/CD pipeline.

Instructions for Examiner to Test the System

Optional Features:

1. Test individual services such as:
 - Change state to INIT, RUNNING, and PAUSED.
 - Fetch a request
 - Retrieve run logs
2. Implement the static analysis step in the CI/CD pipeline.
3. Attempt deployment to an external server and verify its accessibility:
 - Deployment URL: <http://86.50.168.146/>.

Manual Testing Instructions

You can perform manual testing of the system using the following curl commands. These commands allow you to change the system state and interact with it directly.

Change System State

1. **Initialize the system (set state to INIT):**

```
curl localhost:8197/state -X PUT -d "INIT" -H "Content-Type: text/plain" -H "Accept: text/plain"
```

2. Set state to RUNNING:

```
curl localhost:8197/state -X PUT -d "RUNNING" -H "Content-Type: text/plain" -H "Accept: text/plain"
```

3. Set state to PAUSED:

```
curl localhost:8197/state -X PUT -d "PAUSED" -H "Content-Type: text/plain" -H "Accept: text/plain"
```

System Interactions

1. Send a request to the system:

```
curl localhost:8197/request
```

2. Retrieve the system's run log:

```
curl localhost:8197/run-log
```

Data About the Development Platform

- **Hardware:** General-purpose development machine.
- **CPU Architecture:** AMD_64.
- **Operating System:** Windows.
- **Docker Version:** Docker version 27.3.1, build ce12230
- **Docker-Compose Version:** Docker Compose version v2.29.7-desktop.1

Description of the CI/CD Pipeline

Steps in the Pipeline:

1. Version Management:

- GitHub is used to store the project code.
- GitLab is utilized for the CI/CD pipeline.
- Version control follows a branch-based workflow with a dedicated project branch for pipeline execution.

2. Building Tools:

- Commands: docker-compose build && docker-compose up

3. Testing:

- Build and run tests: `docker build -t tests-image ./tests && docker run --rm tests-image`
 - Tools are used to ensure Docker is operational before running tests.
4. **Packing:**
- Application is containerized using Docker.
5. **Deployment:**
- Application is deployed locally
6. **Operating and Monitoring:**
- Basic endpoint monitoring through manual testing using curl.
 - Logs accessible via `/run-log` endpoint.

Example Runs of the Pipeline

Failing Test Log:

- The test stage fails because the service cannot find the correct connection to the application.

Passing Pipeline Log:

- Successfully completes the build, static analysis and deploy stages.

Reflections

Main Learnings:

- Building a robust CI/CD pipeline requires detailed configuration, particularly when deploying to external servers.
- Static analysis integration improves code quality and pre-emptively detects issues.

Worst Difficulties:

- Identifying and resolving connectivity issues related to `DOCKER_HOST` consumed significant time.

- Debugging failed tests in the pipeline was challenging due to limited error information.

Suggestions for Improvement:

- Allocate more time for troubleshooting connectivity and deployment issues.
- Enhance logging for better debugging of pipeline failures.
- Utilize automated monitoring tools to supplement manual checks.

Estimated Effort

- Total hours spent: **250 hours**
 - **Pipeline configuration and debugging:** Majority of the time (~225 hours).
 - **Feature development and testing:** ~25 hours.